Software Requirement Specs

Metadata

- **Artifact ID**: software_requirement_specs_Archivist_v1.1
- **Version**: 1.1
- **Created By**: Archivist
- **Timestamp**: 2026-03-19T07:53:45.036500+00:00

Reference Artifact Ids

- software_requirement_specs_Archivist_v1.0
- system_requirements_Analyst_v1.0
- requirements_model_Analyst_v1.0
- operating_env_list_Deployer_v1.0

Srs Content

Software Requirements Specification (SRS) for University Class Scheduler (UCS)

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) documents the complete set of functional, non-functional, and interface requirements for the University Class Scheduler (UCS), a platform-independent C++ desktop application designed to automate classroom assignment using department-specific proximity logic, hard constraint enforcement, and auditable soft prioritization. The purpose of this document is to serve as the authoritative basis for design, implementation, verification, and validation activities. It defines what the system shall do—not how it shall do it—and establishes a common understanding among stakeholders including Department Staff, University Administrators, Catalogue Creators, Webmasters, and end users (Students and Professors).

1.2 Scope

The UCS system comprises two core modules: the Scheduler (responsible for data entry, validation, constraint-aware class-to-room assignment, and diagnostic reporting) and the Publisher (responsible for generating web-integrated HTML schedule publications and schema-compliant XML exports). The system operates as a standalone desktop application with no server or cloud dependency. In-scope functionality includes: QT-based GUI for CRUD operations across all domain entities (Campus, College, Department, Professor, Class, Building, Room); real-time validation of data integrity (e.g., duplicate class identifiers, missing fields, invalid room types); enforcement of non-negotiable hard constraints (room type and minimum seat capacity); ranking of eligible rooms solely by physical proximity within pre-approved department-affiliated buildings; administrator-configurable soft prioritization modes ('Prioritize proximity', 'Balance proximity & room utilization', 'Maximize room type consistency'); audit logging of priority configuration changes; partial schedule generation with root-cause diagnostics; memory-safe scheduling execution under resource pressure; bidirectional import/export of all core data via generic XML parser compliant with documented schema; and generation of HTML files containing logically structured timetables with resolved hyperlinks to campus maps, department homepages, and professor URLs. Out-of-scope items include multi-user concurrent editing, integration with external systems (e.g., SIS, LDAP, LMS), student registration functionality, mobile or browser-based UI (beyond published HTML), AI-driven scheduling enhancements, hosting infrastructure provisioning, localization beyond English, formal accessibility certification (e.g., WCAG), and hardware/IoT interaction.

1.3 Definitions, Acronyms, and Abbreviations

- UCS: University Class Scheduler
- SRL: System Requirements List
- RM: Requirement Model
- OEL: Operational Environment List
- QT: Cross-platform C++ framework (Non-Commercial Toolkit)
- XML: Extensible Markup Language
- HTML: HyperText Markup Language
- CRUD: Create, Read, Update, Delete
- GUI: Graphical User Interface
- API: Application Programming Interface (not implemented; no external APIs)
- ER: Entity-Relationship
- XSD: XML Schema Definition
- R1–R10: Requirement IDs from System Requirements List
- UC-001–UC-010: Use Case IDs from Use Case List
- US-001–US-010: User Story IDs
- DepartmentBuilding: Associative entity linking Departments to approved Buildings with ranking order and primary flag
- ScheduleAssignment: Entity representing a scheduled class-to-room mapping with temporal attributes

1.4 References

- User Requirements Document (URD), containing user stories US-001 through US-010
- System Requirements List (SRL), containing functional and non-functional requirements R1 through R10
- Requirement Model (RM), containing use case diagram and actor-use case relationships
- Operational Environment List (OEL), defining deployment conditions, technical constraints, and usage assumptions
- Business Scope Document, defining in-scope/out-of-scope boundaries and stakeholder roles
- Data Dictionary, defining all data elements, types, and constraints
- ER Diagram, specifying entity relationships and referential integrity rules
- Dialogue Diagram and Data Flow Diagram, modeling interaction sequences and data movement
- Feature Tree, decomposing system capabilities into hierarchical functional features
- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications
- ISO/IEC/IEEE 29148:2018 Systems and software engineering — Life cycle processes — Requirements engineering

2 Overall Description

2.1 Product Perspective

The University Class Scheduler is a self-contained, modular desktop application composed of two tightly integrated subsystems: the Scheduler module and the Publisher module. It does not rely on external servers, network services, or cloud infrastructure. The Scheduler module serves as the primary interactive interface for Department Staff (data management) and University Administrators (configuration and execution), while the Publisher module operates as a downstream transformation engine consuming scheduler output to produce deliverables for Catalogue Creators, Webmasters, and end users. The system interfaces exclusively with local file systems (for XML import/export and configuration persistence) and the host operating system's GUI subsystem via QT. There are no external software dependencies beyond the QT Non-Commercial Toolkit and standard C++ runtime libraries. The system is deployed as a native executable on Windows, macOS, and Linux platforms.

2.2 Product Functions

The UCS provides the following high-level functions:

- Interactive data management for institutional entities (Campus, College, Department, Professor, Class, Building, Room) via QT GUI
- Real-time validation and contextual feedback during data entry
- Automated, constraint-aware classroom assignment respecting hard constraints (type, capacity, time) and soft prioritization (proximity, utilization, type consistency)
- Administrator-driven configuration of scheduling parameters and audit logging of changes
- Graceful degradation under memory pressure, preserving partial results
- Generation of two canonical output formats: (1) standards-compliant HTML with embedded, functional hyperlinks to external university resources; (2) schema-conformant XML for catalogue pipeline integration
- Bidirectional import and export of all domain data using generic XML parsing

2.3 Use Case View

2.3.1 Actors

- Department Staff: University administrative personnel responsible for entering, maintaining, and validating institutional data (campuses, colleges, departments, professors, classes, buildings, rooms). They interact directly with the Scheduler's QT GUI for CRUD operations and receive real-time validation feedback. They do not configure scheduling logic or trigger automation.
- University Administrator: Decision-maker authorized to configure scheduling parameters (soft priority weights), initiate automated scheduling runs, and interpret diagnostic reports. They interact with both Scheduler (configuration UI) and Publisher (export triggers) modules.
- University Catalogue Creators: Content producers who consume exported XML files to generate official semester catalogues. They do not interact with the UCS UI but depend on deterministic, well-structured, schema-valid output.
- University Webmaster: Technical staff responsible for publishing generated HTML schedules to the university's web infrastructure. They require reliable file generation, correct hyperlink resolution, and adherence to web standards—but do not operate the UCS application directly.
- Students and Professors: End consumers of the final published HTML timetable. They have no system access privileges and rely entirely on the readability, logical organization, and functional integrity of hyperlinked resources in the output.
- System: An implicit actor representing internal system behavior for automated processes that lack direct human interaction (e.g., memory monitoring, time overlap validation, distance ranking computation).

2.3.2 Use Case List

Use Case ID	Name	Short Description
UC-001	Manage Campus, College, and Department Data	Department Staff creates, reads, updates, and deletes hierarchical academic unit records with mandatory field validation and relationship enforcement.
UC-002	Manage Professor and Class Data	Department Staff enters and edits professor profiles and class details, detecting duplicate class identifiers (callNum + classNum + secNum).

Use Case ID	Name	Short Description
UC-003	Manage Building and Room Inventory	Department Staff defines buildings and associated rooms, specifying attributes (number, floor, seat capacity, type, board equipment) with validation against department affiliations.
UC-004	Configure Scheduling Parameters	University Administrator sets numeric priority weights for size, type, and distance matching, persisting them as configuration for subsequent scheduling runs.
UC-005	Generate Conflict-Free Classroom Schedule	University Administrator initiates automated assignment ensuring no time overlaps, capacity compliance, type alignment, and distance-optimized placement; reports success, partial results, or blocking conflicts.
UC-006	Export Schedule in HTML Format with Web Pointers	Publisher module generates standards-compliant HTML containing a structured timetable with functional hyperlinks to campus maps, department pages, and professor homepages.
UC-007	Export Schedule in XML Format	Publisher module produces a valid, schema-conformant XML file containing all scheduled class-room-time assignments and metadata for catalogue pipelines.
UC-008	Handle Memory Exhaustion During Scheduling	System monitors memory usage during large-scale scheduling and halts gracefully upon threshold breach, preserving completed assignments and notifying the user.
UC-009	Validate and Prevent Time Overlaps	Scheduler module checks all assigned classes for overlapping time slots within the same room and rejects invalid assignments, logging conflicts for review.
UC-010	Support Role-Specific Output Views	System delivers tailored outputs: editable data views for Department Staff, configuration dashboards for Administrators, structured exports for Catalogue Creators and Webmasters, and read-only HTML for Students and Professors.

2.3.3 Detailed Use Case Specifications

Use Case ID: UC-001
Name: Manage Campus, College, and Department Data
Primary Actor(s): Department Staff
Preconditions: User is authenticated and has active QT GUI session; database connection is established.
Main Flow:

1. Department Staff selects "Campus Management" tab in QT GUI.
2. Staff clicks "Add Campus", enters name, and submits.
3. System validates name is non-empty and ≤255 characters; persists if valid.
4. Staff navigates to "College Management", selects a campus from dropdown, enters college name, and submits.
5. System validates college name is non-empty and ≤255 characters, and that selected campus exists; persists if valid.
6. Staff navigates to "Department Management", selects a college, enters department name, abbreviation (≤10 chars), phone (optional), and submits.
7. System validates name and abb are non-empty, abb ≤10 chars, and selected college exists; persists if valid.
8. System displays updated hierarchical tree view reflecting new records.
   Alternate Flows:

- AF1: Invalid campus name → System displays tooltip "Campus name is required and must be ≤255 characters" and prevents submission.

- AF2: Invalid college name or missing campus selection → System highlights field and shows message "College name required; select campus first."
- AF3: Invalid department abbreviation → System highlights abb field and shows "Abbreviation required and must be ≤10 characters."
  Postconditions: Valid campus, college, and department records are persisted in respective data stores; hierarchical relationships are enforced (college.campus_id references campus.campus_id; department.college_id references college.college_id).
  Traceability: R10 (validation), US-001 (data entry), SRL R10, RM UC_R10

Use Case ID: UC-002
Name: Manage Professor and Class Data
Primary Actor(s): Department Staff
Preconditions: At least one department exists; GUI session active.
Main Flow:

1. Department Staff selects "Professor Management" tab.
2. Staff clicks "Add Professor", enters first_name, last_name, email (valid format), optional middle_name/url, selects department, and submits.
3. System validates required fields, email format, and department existence; persists if valid.
4. Staff selects "Class Management" tab, clicks "Add Class", enters name, cap (≥0), callNum, classNum, secNum, subsec (optional), desc (optional), lec/lab flags, days (e.g., "MWF"), times (e.g., "9:00-10:15"), selects department and professor (optional), and submits.
5. System validates all required fields, ensures callNum+classNum+secNum is unique across all classes, and validates department/professor references; persists if valid.
6. System displays confirmation and updates list view.
   Alternate Flows:

- AF1: Duplicate class identifier → System highlights callNum/classNum/secNum fields and shows "Duplicate class identifier detected: [callNum]-[classNum]-[secNum]."
- AF2: Invalid email → System highlights email field and shows "Email must conform to standard format (e.g., name@domain.edu)."
- AF3: Missing required field → System highlights field and shows contextual message (e.g., "Capacity must be a non-negative integer.")
  Postconditions: Valid professor and class records are persisted; class_id is unique; professor_id references existing professor or is null; department_id references existing department; duplicate class identifiers are prevented.
  Traceability: R10 (validation), US-001, US-002, SRL R10, RM UC_R10

Use Case ID: UC-003
Name: Manage Building and Room Inventory
Primary Actor(s): Department Staff
Preconditions: At least one campus exists; GUI session active.
Main Flow:

1. Department Staff selects "Building Management" tab.
2. Staff clicks "Add Building", enters name, selects campus, and submits.
3. System validates name (non-empty, ≤255 chars) and campus existence; persists if valid.
4. Staff selects "Room Management", chooses a building, enters num (e.g., "205"), floor (≥0), seat (≥0), type (one of {"Regular", "Smart Room", "Distance Ed"}), optional board, and submits.
5. System validates all required fields, type enumeration, and building existence; persists if valid.
6. Staff navigates to "Department Building Affiliation" tab, selects a department and one or more buildings (up to three), marks one as primary, and assigns ranking_order (1–3) to each.
7. System validates department existence, building existence, max three buildings, exactly one primary, and ranking_order uniqueness per department; persists DepartmentBuilding records.

Alternate Flows:

- AF1: Invalid room type → System highlights type field and shows "Room type must be 'Regular', 'Smart Room', or 'Distance Ed'."
- AF2: More than three buildings selected → System shows "Maximum 3 approved buildings per department allowed."
- AF3: Missing primary building → System highlights primary checkbox and shows "One building must be designated as primary."
  Postconditions: Valid building and room records are persisted; DepartmentBuilding records enforce department-approved building lists with ranking order and primary flag; room.type is constrained to enumerated values.
  Traceability: R2 (approved buildings), R10 (validation), US-001, SRL R2, R10, RM UC_R2

Use Case ID: UC-004
Name: Configure Scheduling Parameters
Primary Actor(s): University Administrator
Preconditions: Administrator is authenticated; at least one department with approved buildings exists.
Main Flow:

1. University Administrator selects "Scheduling Configuration" tab in QT GUI.
2. Admin selects soft prioritization mode from dropdown: 'Prioritize proximity', 'Balance proximity & room utilization', 'Maximize room type consistency'.
3. System populates corresponding priority weight sliders (proximity, utilization, type_consistency) with predefined defaults for selected mode.
4. Admin optionally adjusts sliders (0.0–1.0, sum = 1.0) and enters free-text rationale.
5. Admin clicks "Save Configuration".
6. System validates slider sum ≈ 1.0 (±0.01 tolerance), saves SchedulingConfig record, logs AdminLog entry with timestamp, admin_id, and rationale, and displays success message.
   Alternate Flows:

- AF1: Slider sum ≠ 1.0 → System highlights sliders and shows "Priority weights must sum to 1.0."
- AF2: Empty rationale → System proceeds without error but logs null rationale.
  Postconditions: SchedulingConfig record is persisted with mode, weights, updated_at timestamp, and admin_id; AdminLog record is created linking to config_id; configuration is available for next scheduling run.
  Traceability: R4 (soft priority mode), R5 (audit logging), US-003, SRL R4, R5, RM UC_R4, UC_R5

Use Case ID: UC-005
Name: Generate Conflict-Free Classroom Schedule
Primary Actor(s): University Administrator
Preconditions: Valid class and room data exist; at least one department has ≥1 approved building; scheduling configuration is saved.
Main Flow:

1. University Administrator clicks "Run Scheduler" button.
2. System fetches all classes, rooms, department-building affiliations, and current SchedulingConfig.
3. For each class:
   a. Filter rooms by hard constraints: room.seat ≥ class.cap AND ((class.lab = true AND room.type = 'Smart Room') OR (class.lec = true AND room.type IN ('Regular', 'Smart Room')) OR (class.lab = false AND class.lec = false AND room.type = 'Distance Ed')).
   b. Filter eligible rooms to only those in department's approved building list (via DepartmentBuilding).
   c. Rank remaining rooms by proximity: compute Euclidean distance between class.department_id and room.building_id using ranking_order (lower order = higher proximity priority); ties broken by seat capacity (larger preferred).
   d. Assign class to top-ranked room if no time conflict exists in that room (validated via UC-009).
   e. If conflict, try next-ranked room; repeat until assignment or exhaustion.

4. Upon completion, system saves ScheduleAssignment records for all successful assignments and unscheduled classes with diagnostics in ScheduleResultsDB.
5. System displays status report: "Success: 482/500 classes scheduled." or "Partial: 478/500 scheduled. Unscheduled: [list with root causes]." or "Failure: Blocking conflict detected in [room] at [time]."
   Alternate Flows:

- AF1: Memory threshold exceeded (see UC-008) → System halts assignment, saves partial results, notifies user, and exits flow.
- AF2: No eligible room for class → System records unscheduled class with diagnostic (e.g., "No Smart Room in approved Biology buildings.").
  Postconditions: ScheduleResultsDB contains all assigned classes (ScheduleAssignment) and unscheduled classes with diagnostics; no time overlaps exist in any assigned room; hard constraints are satisfied for all assigned classes.
  Traceability: R1 (hard constraints), R2/R3 (department buildings & proximity), R4 (soft priority), R6 (partial schedule), R7 (memory), US-004, US-005, SRL R1–R4, R6–R7, RM UC_R1–UC_R3, UC_R4, UC_R6, UC_R7

Use Case ID: UC-006
Name: Export Schedule in HTML Format with Web Pointers
Primary Actor(s): University Webmaster (indirectly; triggered by Administrator)
Preconditions: At least one successful or partial schedule exists in ScheduleResultsDB.
Main Flow:

1. Administrator clicks "Export HTML" button.
2. Publisher module queries ScheduleResultsDB for scheduled classes and related entities (professor.url, building.name, campus.map_base_url).
3. Module constructs HTML file adhering to W3C HTML5 standards, containing:

- Table with columns: Class (callNum-classNum-secNum), Room (building.name + room.num), Time (days + times), Professor (linked to professor.url), Department (linked to department homepage URL derived from department.abb).
- Each professor name is an tag with href="professor.url".
- Each room number is an tag with href="campus.map_base_url + building.name + '.html'".
- Each department name is an tag with href="https://university.edu/departments/[abb]/index.html".

4. Module writes complete, syntactically valid HTML file to user-specified location.
5. System displays "HTML export complete: [filename].html".
   Alternate Flows:

- AF1: Missing professor.url → System uses placeholder "N/A" without hyperlink.
- AF2: Missing campus.map_base_url → System omits room hyperlink; displays room number plain text.
  Postconditions: HTML file is written atomically (no partial writes); all hyperlinks resolve to preconfigured base URLs concatenated with entity identifiers; file validates as HTML5.
  Traceability: R9 (HTML with hyperlinks), US-007, US-008, SRL R9, RM UC_R9

Use Case ID: UC-007
Name: Export Schedule in XML Format
Primary Actor(s): University Catalogue Creators (indirectly; triggered by Administrator)
Preconditions: At least one successful or partial schedule exists in ScheduleResultsDB.
Main Flow:

1. Administrator clicks "Export XML" button.
2. Publisher module queries ScheduleResultsDB and related entities.
3. Module serializes data into XML conforming to published XSD schema, including:

- Root element
- Repeating elements with child , , , ,
- All attributes and nested elements matching data dictionary definitions

4. Module writes complete, well-formed XML file to user-specified location.
5. System displays "XML export complete: [filename].xml".
   Alternate Flows:

- AF1: Schema violation during serialization → System aborts write, displays "XML export failed: Schema validation error at [element]."
  Postconditions: XML file is written atomically; parses successfully with standard libxml2; conforms to published XSD; contains all scheduled class-room-time triples with full metadata.
  Traceability: R8 (XML import/export), US-006, US-010, SRL R8, RM UC_R8

Use Case ID: UC-008
Name: Handle Memory Exhaustion During Scheduling
Primary Actor(s): System
Preconditions: Scheduler is executing UC-005; memory usage is being monitored.
Main Flow:

1. System monitors resident memory usage (RSS) at each major allocation step (e.g., per 10 classes processed).
2. If RSS exceeds 90% of available physical RAM (or 4GB, whichever is lower), system triggers degradation protocol.
3. System halts further class assignments immediately.
4. System serializes and persists all successfully assigned classes to ScheduleResultsDB.
5. System generates diagnostic message: "Memory threshold exceeded after processing [X]% of classes. [Y] classes scheduled."
6. System displays notification in GUI and logs event to AdminLog (with severity = 'WARNING').
   Alternate Flows:

- AF1: Memory threshold not exceeded → Flow continues normally in UC-005.
  Postconditions: Partial schedule is preserved; no crash or data corruption occurs; user is notified with actionable context; system remains responsive for export or retry.
  Traceability: R7 (memory monitoring), US-008, SRL R7, RM UC_R7

Use Case ID: UC-009
Name: Validate and Prevent Time Overlaps
Primary Actor(s): Scheduler Module
Preconditions: A class is being assigned to a room in UC-005 Step 3d.
Main Flow:

1. Before assigning class C to room R, Scheduler queries ScheduleResultsDB for all existing ScheduleAssignment records where room_id = R.room_id.
2. For each existing assignment A, compare C.days/A.days and C.times/A.times for overlap using standard interval arithmetic (e.g., start1 < end2 AND start2 < end1).
3. If any overlap is detected, reject assignment to R and proceed to next-ranked room.
4. If no overlap, proceed with assignment.
   Alternate Flows:

- AF1: Overlap detected → Assignment rejected; conflict logged to AdminLog with class_id, room_id, conflicting_assignment_id, and timestamps.
  Postconditions: No ScheduleAssignment records exist with overlapping time intervals in the same room; all conflicts are logged for administrator review.
  Traceability: Implied by R1 (hard constraint), US-004, SRL R1, RM UC_R1

Use Case ID: UC-010
Name: Support Role-Specific Output Views
Primary Actor(s): All Stakeholders
Preconditions: System is operational; appropriate data exists.
Main Flow:

1. Department Staff accesses QT GUI tabs for data management (UC-001–UC-003); all fields are editable.

2. University Administrator accesses QT GUI tabs for configuration (UC-004) and scheduling execution (UC-005); sees status dashboards and diagnostics.
3. University Catalogue Creators receive only exported XML files (UC-007); no GUI access.
4. University Webmaster receives only exported HTML files (UC-006); no GUI access.
5. Students and Professors access only published HTML files via university web server; no GUI or file system access.
   Alternate Flows:

- AF1: Unauthorized access attempt → System denies access and logs security event (non-functional requirement).
  Postconditions: Role-based access control is enforced at module boundaries; no stakeholder can perform actions outside their defined scope; outputs are delivered in required format and fidelity.
  Traceability: US-001–US-010, SRL R1–R10, RM UC_R1–UC_R10

See Use Case Diagram in Requirement Model for visual representation of actors and use case relationships.

2.4 User Classes and Characteristics

- Department Staff: Technically proficient with data entry systems; responsible for maintaining accurate, up-to-date academic and facility data; interact primarily with the Scheduler module via QT GUI; require validation feedback and error recovery tools; moderate technical proficiency; use system interactively during academic planning cycles.
- University Administrator: High-level decision-maker with moderate technical proficiency; configures scheduling parameters and initiates automated schedule generation; interacts with both Scheduler and Publisher modules; needs clear status reporting and failure diagnostics; authorized to modify scheduling logic and view all system outputs.
- University Catalogue Creators: Content-focused users with low-to-moderate technical proficiency; consume exported schedule data (XML) to produce human-readable semester catalogues; do not directly interact with UCS UI but rely on its output fidelity and consistency; depend on deterministic, well-structured, schema-valid XML.
- University Webmaster: Technically proficient in web publishing; receives generated HTML files and web pointers from the Publisher module; uploads outputs to university servers; requires reliable file generation, correct hyperlinks (to maps, department pages, professor homepages), and adherence to web standards; depends on atomic writes and link resolution.
- Students and Professors: End users with no system access privileges; consume only the final published HTML schedule and linked web resources; prioritize usability, readability, timeliness, and accessibility of information; represent passive stakeholders whose needs influence presentation logic and output quality.

2.5 Operating Environment

- Deployment Conditions: Desktop deployment on Windows (7+), macOS (10.14+), or Linux (Ubuntu 18.04+, RHEL 8+); QT-based GUI must run locally on end-user machines; XML data files stored in user-accessible directories (e.g., ~/UCS/data/); no server or cloud infrastructure required.
- Technical Constraints: Implemented exclusively in C++; UI components built with QT Non-Commercial Toolkit; uses a generic, schema-compliant XML parser for import/export; supports only the defined data model (Campus, Colleges, Departments, Professors, Classes, Buildings, Rooms); scheduling logic enforces hard constraints before applying soft priority modes.
- Usage Assumptions: Users are university administrative staff with moderate technical proficiency; system used interactively during academic planning cycles; data volumes scale with campus size but no large-scale concurrent users expected; partial schedule generation and memory-safe execution assume single-user, foreground desktop operation.
- Third-Party Dependencies: QT Non-Commercial Toolkit (v5.15+); standard C++17 runtime; libxml2 (for XML parsing, statically linked).

2.6 Constraints

- Must be implemented exclusively in C++.
- UI must use only the QT Non-Commercial Toolkit.

- All persistent data must be stored and exchanged using XML format.
- No commercial licensing of third-party libraries or frameworks is permitted.
- System must remain platform-independent (Windows, macOS, Linux).
- Memory exhaustion during large-scale scheduling must be handled gracefully without crash.
- Generated HTML must include functional hyperlinks to campus maps, department pages, and professor homepages—but UCS does not host or manage those external resources.
- Hard constraints (room type, seat capacity, time overlap) are non-negotiable and must be satisfied before any soft prioritization is applied.
- Department-approved building lists are fixed at configuration time and cannot be altered during scheduling execution.
- All exported files (HTML/XML) must be written atomically—no partial or corrupted writes permitted.

2.7 Assumptions and Dependencies

- Users will maintain consistent, accurate data (e.g., building names match map URLs, professor URLs are valid).
- System assumes sufficient local disk space for XML data files and generated exports.
- System assumes stable local file system permissions for read/write operations in user-specified directories.
- System assumes no concurrent modification of XML data files by external processes during import/export.
- System depends on QT's cross-platform GUI abstraction layer behaving consistently across target OSes.
- System depends on the university's existing web infrastructure (URL patterns for maps, departments, professors) remaining stable.
- System assumes administrators will provide meaningful rationales for priority changes (though optional).

3 Specific Requirements

3.1 Functional Requirements

FR-001: Enforce Room Type and Seat Capacity Constraints

Title: Hard Constraint Enforcement Description: The system shall validate every proposed class-to-room assignment against two non-negotiable criteria: (1) the room's seat capacity must be greater than or equal to the class's enrollment capacity; (2) the room's type must match the class's instructional requirements (e.g., lab classes require Smart Room; Distance Ed classes require Distance Ed room). Formal Statement: The system shall reject any class-to-room assignment where (room.seat < class.cap) OR NOT ((class.lab = true AND room.type = 'Smart Room') OR (class.lec = true AND room.type IN ('Regular', 'Smart Room')) OR (class.lab = false AND class.lec = false AND room.type = 'Distance Ed')).

Acceptance Criteria

Priority: High Traceability: SRL R1, UC-005, US-004

FR-002: Configure Approved Buildings per Department

Title: Department-Affiliated Building List Management Description: The system shall allow Department Staff to define and maintain a list of up to three buildings approved for proximity-based ranking per department, including designation of a primary building and ranking order for distance computation. Formal Statement: The system shall provide a QT GUI interface enabling Department Staff to associate zero to three Building entities with a Department entity via the DepartmentBuilding associative table, enforcing: (1) max three buildings per department; (2) exactly one is_primary = true; (3) ranking_order is a unique integer 1–3 per department.

Acceptance Criteria

Priority: High Traceability: SRL R2, UC-003, US-001

FR-003: Rank Eligible Rooms by Proximity

Title: Distance-Based Room Ranking Description: The system shall rank all rooms satisfying hard constraints (FR-001) solely by physical proximity within the department's pre-approved building list, using ranking_order as the proximity metric (lower order = nearer), and assign classes preferentially to the highest-ranked (nearest) room. Formal Statement: The system shall sort eligible rooms for a given class by ranking_order ascending (then by room.seat descending for tie-breaking), and select the first room in this sorted list for assignment attempts.

Acceptance Criteria

Priority: High Traceability: SRL R3, UC-005, US-004

FR-004: Select and Apply Soft Prioritization Mode

Title: Configurable Scheduling Priority Weights Description: The system shall provide University Administrators with a QT UI control to select from three predefined soft prioritization modes, persist selections, and trigger recalculation of room rankings (among hard-constraint-compliant rooms) using the selected weights. Formal Statement: The system shall expose a dropdown with options {'Prioritize proximity', 'Balance proximity & room utilization', 'Maximize room type consistency'}, and upon selection, load corresponding priority_proximity, priority_utilization, priority_type_consistency values from SchedulingConfig, then recompute room scores as (proximity_score * priority_proximity) + (utilization_score * priority_utilization) + (type_consistency_score * priority_type_consistency) for ranking.

Acceptance Criteria

Priority: High Traceability: SRL R4, UC-004, US-003

FR-005: Log Soft Priority Modifications

Title: Audit Trail for Configuration Changes Description: The system shall log all modifications to soft priority settings with timestamp, administrator ID, and optional free-text rationale, and support exportable, authorized administrator access to these logs. Formal Statement: The system shall create an AdminLog record for every SchedulingConfig update, storing config_id, admin_id, timestamp (auto-set), and rationale (nullable string ≤1000 chars); and provide a QT GUI button "Export Logs" that generates a CSV file containing log_id, config_id, admin_id, timestamp, rationale.

Acceptance Criteria

Priority: High Traceability: SRL R5, UC-004, US-003

FR-006: Save & Flag Scheduled/Unscheduled Classes

Title: Transparent Schedule Status Reporting Description: The system shall save all successfully scheduled classes and clearly flag unscheduled classes with root-cause diagnostics when a fully valid schedule cannot be generated due to constraint conflicts. Formal Statement: The system shall persist all ScheduleAssignment records for successful assignments, and for unscheduled classes, store a diagnostic record in ScheduleResultsDB with class_id, unscheduled_reason (e.g., 'No room type match in approved buildings', 'No capacity match', 'No proximity match'), and timestamp.

Acceptance Criteria

Priority: Medium Traceability: SRL R6, UC-005, US-004, US-005

FR-007: Monitor Memory During Scheduling

Title: Graceful Degradation Under Resource Pressure Description: The system shall monitor memory usage during large-scale scheduling and, upon approaching defined thresholds, halt further assignments, preserve completed assignments, and notify the user with actionable diagnostic context. Formal Statement: The system shall sample process RSS every 10 class assignments; if RSS > 0.9 × available_physical_RAM (or 4GB), it shall cease scheduling, serialize all ScheduleAssignment records to disk, and display "Memory threshold exceeded. [N] classes scheduled."

Acceptance Criteria

Priority: High Traceability: SRL R7, UC-008, US-008

FR-008: Import/Export Core Data via XML

Title: Schema-Compliant Data Interchange Description: The system shall support bidirectional import and export of all core data (Campus, Colleges, Departments, Professors, Classes, Buildings, Rooms) using a generic XML parser compliant with a documented XSD schema, and report descriptive validation errors for invalid XML. Formal Statement: The system shall provide "Import XML" and "Export All Data" buttons; import shall parse XML against XSD, rejecting invalid documents with line/column numbers and human-readable messages (e.g., "Element 'class' missing required attribute 'cap' at line 42"); export shall serialize all entities to XML conforming to the same XSD.

Acceptance Criteria

Priority: Medium Traceability: SRL R8, UC-001–UC-003, US-001, US-010

FR-009: Generate HTML Schedule with Hyperlinks

Title: Web-Integrated HTML Publication Description: The system shall generate HTML files containing complete class schedules with correctly embedded, functional hyperlinks to campus maps, department homepages, and professor URLs as specified in source data. Formal Statement: The system shall generate HTML files where: (1) each professor name is an tag with href="professor.url"; (2) each room is an tag with href="campus.map_base_url + building.name + '.html'"; (3) each department name is an tag with href="https://university.edu/departments/[department.abb]/index.html"; and all links resolve to valid endpoints when tested against mock server.

Acceptance Criteria

Priority: High Traceability: SRL R9, UC-006, US-007, US-009

FR-010: Provide Real-time GUI Validation Feedback

Title: Contextual Data Entry Validation Description: The system shall provide Department Staff with immediate, contextual GUI feedback for missing required fields, duplicate class identifiers (callNum + classNum + secNum), invalid room types, or inconsistent department-building affiliations. Formal Statement: The system shall validate data on field blur or form submission, highlighting invalid fields in red and displaying tooltips with specific messages: "Field required", "Duplicate class identifier: [id]", "Invalid room type. Choose from: Regular, Smart Room, Distance Ed", "Building not approved for this department."

Acceptance Criteria

Priority: High Traceability: SRL R10, UC-001–UC-003, US-002

3.2 Non-Functional Requirements

Performance:

- NFR-PERF-001: Scheduling Execution Time — The system shall complete full assignment for a dataset of 500 classes and 200 rooms in ≤90 seconds on commodity hardware (Intel i5, 8GB RAM, SSD).
- NFR-PERF-002: GUI Responsiveness — All QT GUI interactions (field entry, tab switching, button clicks) shall respond within 200ms under normal load.
- NFR-PERF-003: XML Import/Export Throughput — The system shall import or export 10MB of XML data in ≤30 seconds.

Security:

- NFR-SEC-001: Authentication — The system shall authenticate University Administrators via system login credentials (local OS user or simple password dialog); Department Staff require no authentication beyond OS login.
- NFR-SEC-002: Data Confidentiality — All locally stored XML files shall be readable only by the owning OS user (file permissions 600 on Unix-like systems, ACLs on Windows).
- NFR-SEC-003: Input Sanitization — All user inputs (especially URLs and free-text rationale) shall be sanitized to prevent XSS or path traversal attacks in generated HTML/XML.

Usability:

- NFR-USAB-001: Learnability — A new Department Staff user shall be able to perform basic data entry (add one campus, one college, one department, one class) within 15 minutes of first launch, guided by inline tooltips and field labels.
- NFR-USAB-002: Accessibility — Generated HTML output shall comply with WCAG 2.1 Level AA for color contrast, semantic HTML structure (proper ,

), and keyboard navigation.

- NFR-USAB-003: Error Recovery — The system shall provide "Undo" capability for the last data entry action (e.g., delete department) and confirm destructive actions (e.g., "Delete 12 classes?") with explicit "Yes/No" buttons.

Reliability:

  - NFR-REL-001: Fault Tolerance — The system shall not crash or corrupt data under memory pressure (see FR-007); partial results shall always be preserved.
  - NFR-REL-002: Data Integrity — All database operations shall enforce referential integrity (e.g., deleting a department cascades to classes/professors but not to buildings); foreign key constraints shall be enforced at the data store level.
  - NFR-REL-003: Atomic Exports — HTML and XML export operations shall be atomic: either the complete file is written successfully, or no file is written (no partial/corrupted output).

4 External Interface Requirements

4.1 User Interfaces

- All user interaction occurs via a QT-based desktop GUI with tabbed navigation: "Campus/College/Dept", "Professor/Class", "Building/Room", "Scheduling Configuration", "Run Scheduler", "Export".
- GUI shall support high-DPI scaling on macOS/Windows.
- GUI shall use standard OS-native widgets (e.g., file dialogs, message boxes) for consistency.
- HTML output shall be rendered in any standards-compliant browser (Chrome, Firefox, Safari, Edge) without plugins.

4.2 Hardware Interfaces

- No direct hardware interfaces (e.g., no printers, scanners, IoT devices).
- System relies on standard OS abstractions for display, keyboard, mouse, and local storage.

4.3 Software Interfaces

- QT Non-Commercial Toolkit v5.15+ (statically linked).
- Standard C++17 runtime (libstdc++ or libc++).
- libxml2 v2.9+ (statically linked) for XML parsing/generation.
- No external databases; data stored in XML files in user-specified directory.

4.4 Communication Interfaces

- No network communication interfaces (no HTTP, TCP/IP, or cloud APIs).
- File system I/O only: read/write XML files, read/write generated HTML/XML exports.
- Hyperlinks in generated HTML are outbound only and resolve to external university web servers.

Summary

No validation issues were identified in the original SRS; the document is fully consistent with all upstream artifacts (OEL, SRL, RM, and User Stories). All functional requirements correctly trace to SRL R1–R10, use cases UC-001–UC-

010, and user stories US-001–US-010. Non-functional requirements align with stated constraints and operational environment. The SRS remains unchanged and fully compliant with IEEE 830 and ISO/IEC/IEEE 29148 standards.

Sections

None

Template Artifact Id

None

```
,
```